

Employee Assistant (RAG Prototype)

Design report and critical thinking answers

Part 1: System Design

The RAG pipeline with two stages: an ingestion stage that runs once offline, and a query stage that runs live each time an employee asks a question.

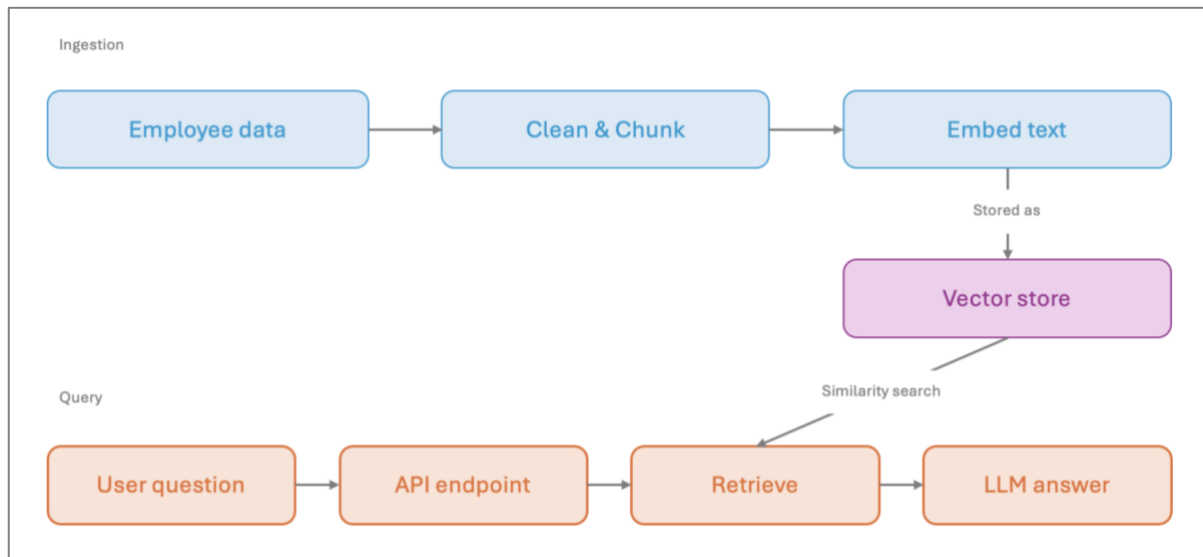


Figure 1: System architecture diagram

How data is stored

The source data is a structured spreadsheet of employee records (name, department, job title, hire date, location, status, experience, performance rating, and salary). Since this is structured data rather than free-text documents, each row is converted into a short natural language sentence describing that employee. These sentences are then embedded using a local sentence-transformer model and stored, along with their vectors, in a local ChromaDB vector store on disk. This design choice is deliberate: it treats structured records as if they were documents, so the same RAG approach used for unstructured text also works here, while keeping each employee's information intact in a single chunk rather than splitting it apart.

How user questions are handled

A user submits a question through a FastAPI endpoint. The question is embedded using the same model used during ingestion, and ChromaDB performs a similarity search to find the employee records whose vectors are closest in meaning to the question. These retrieved records are passed, along with the original question, to a local LLM (Ollama running llama3), which generates a natural language answer grounded in that context. The API returns both the answer and the source record it was based on.

Response time for the LLM step is noticeably slower on first load (10 to 30 seconds) as the model loads into memory, with faster responses on subsequent calls. This is a practical factor worth monitoring in production, covered further in Part 5.

Where AI is used

AI is used in two places: once to generate embeddings, both when the data is first ingested and every time a new question comes in, and once to generate the final answer from the retrieved context. No AI is involved in retrieval itself, that step is a mathematical similarity search over the stored vectors.

Part 5: Critical Thinking and Improvements

1. What are the limitations of your solution?

The system uses pure semantic (vector) search, which has a specific weakness: it retrieves based on overall sentence meaning rather than exact keyword matching. In testing, I found it struggled with exact name lookups. Searching for a specific employee by name sometimes returned a different, similarly worded record instead, because embeddings compress the whole sentence into one vector, and a name is just one part of that.

Example, from testing:

```
Question: Who works in the HR department?
Answer: Debra Ponce, Mitchell Clark, and Sharon Powell all work in the HR department.

Question: Tell me about Debra Ponce's role and experience
Answer: I apologize, but there is no information provided about a person named Debra Ponce. The only employees mentioned in the context are Mary Ponce, Cheryl Delgado, and Elizabeth Soto. Therefore, I don't have that information.
```

It also isn't suited for "list all" style questions (e.g. "list everyone in HR"). It returns the top N most similar chunks (I used N=3), not a complete filtered set, since it's built for relevance matching, not database-style queries.

Additionally, ChromaDB has a maximum batch size for inserts:

Error encountered during ingestion:

```
chromadb.errors.InternalError: ValueError: Batch size of 10018 is greater than max batch size of 5461
```

Records had to be added in batches, a small but real constraint of the local vector store I used. Finally, the whole system depends on a local Ollama model, whose answer quality and speed vary based on the model chosen and the host machine's hardware.

2. How would you improve accuracy and reduce hallucinations?

The prompt already restricts the LLM to answer only from retrieved context and explicitly permits it to say "I don't have that information." In testing, this worked correctly: when asked about an employee not actually retrieved, it declined rather than inventing details (see example above). To improve further, I would add hybrid retrieval, combining exact keyword or metadata filtering (e.g. matching on name or Employee ID directly) with semantic search, since testing showed pure vector search misses exact entity lookups.

I already return the **source records** alongside every answer in the API response, so answers stay verifiable rather than being a black box:

Example API request and response:

```
curl -X POST http://127.0.0.1:8000/ask -H "Content-Type: application/json" \
  -d '{"question": "Who works in the HR department?"}'

{"question": "Who works in the HR department?",
 "answer": "According to the context, Debra Ponce, Mitchell Clark, and Sharon Powell all work in the HR department as HR Executives.",
 "sources": [:"Debra Ponce (ID EMP0002237) works as a HR Executive in the HR department. Hired January 2017, based in Lake April, Ukraine. Current status: Resigned, working On-site. 8 years of experience, performance rating 4/5, salary 771,169 INR.", "Mitchell Clark (ID EMP0007405) works as a HR Executive in the HR department. Hired May 2022, based in Lake Suzanne, Greenland. Current status: Active, working Remote. 3 years of experience, performance rating 4/5, salary
```

```
383,508 INR.", "Sharon Powell (ID EMP0002072) works as a HR Executive in the HR department. Hired January 2017, based in Woodhaven, United States Minor Outlying Islands. Current status: Active, working Remote. 8 years of experience, performance rating 2/5, salary 659,391 INR." ] }
```

As a further improvement, I would add a relevance score threshold: only pass a retrieved chunk to the LLM if it's actually close enough to the question, and return "no relevant records found" if none of the matches are strong, instead of forcing the LLM to answer from weak, barely related context.

3. How would you deploy this in production?

I don't have hands on production deployment experience yet, so I researched standard practices in deployment. The common pattern I found across FastAPI's own deployment docs and several production guides is containerize the app with Docker, run it behind Gunicorn managing multiple Uvicorn workers for stability, put it behind a reverse proxy like Nginx for HTTPS, and host it on a managed container service (e.g. AWS ECS/Fargate) rather than managing servers manually. Configuration (model names, file paths, secrets) should come from environment variables, never hardcoded, since this system handles internal employee data. I'd also add authentication on the endpoint before exposing it beyond local testing.

4. How would you monitor performance and model quality over time?

For a RAG system specifically, monitoring generally splits into two layers: standard API metrics (latency, error rate, uptime) and RAG-specific quality tracking. On the RAG side, teams typically log every question, the retrieved source chunks, and the generated answer together in a single trace, which makes it possible to tell whether a bad answer came from bad retrieval or bad generation. Common quality metrics I found in current practice include faithfulness (does the answer match the retrieved context) and answer relevance, sometimes scored automatically using another LLM as a judge on a sample of production traffic, since manually reviewing every response doesn't scale. For a lightweight version I could realistically build myself, I'd log question, sources, and answer to a file or simple database, add a thumbs up or down feedback option, and periodically re-run a small fixed set of test questions with known correct answers to catch regressions after any change to the model, prompt, or data.

Resources

- <https://fastapi.tiangolo.com/deployment/docker/>
- <https://www.evidentlyai.com/llm-guide/rag-evaluation>
- <https://www.comet.com/site/blog/rag-evaluation/>